



TECHNICAL UNIVERSITY

OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

Real-Time Light Dosimetry for Heritage Digital Twins via Texture Space Ray Tracing

LICENSE THESIS

Graduate: **Sebastian Duică**

Supervisor: **Assoc. Prof. Victor Ioan Băcu**

2026



TECHNICAL UNIVERSITY

OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

DEAN,
Prof. dr. ing. Vlad MUREȘAN

HEAD OF DEPARTMENT,
Prof. dr. ing. Rodica POTOLEA

Graduate: **Sebastian Duică**

Real-Time Light Dosimetry for Heritage Digital Twins via Texture Space Ray Tracing

1. **Project proposal:** Development of ChronoLux, an interactive Digital Twin for real-time light dosimetry using Texture Space Ray Tracing in DirectX 12.
2. **Project contents:** Introduction, Project Objectives, Bibliographic Research, Analysis and Theoretical Foundation, Detailed Design and Implementation, Testing and Validation, User's Manual, Conclusions, Bibliography.
3. **Place of documentation:** Technical University of Cluj-Napoca, Computer Science Department
4. **Consultants:**
5. **Date of issue of the proposal:** October 1, 2025
6. **Date of delivery:** July 10, 2026

Graduate: _____

Supervisor: _____

**TECHNICAL UNIVERSITY**

OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE**Declarație pe proprie răspundere privind
autenticitatea lucrării de licență**

Subsemnatul(a) _____, _____ legitimat(ă) cu

_____ seria _____ nr. _____
(conform și copiei anexate prezentei declarații, copie sem-
nată și certificată "conform cu originalul"), autorul lucrării

elaborată în vederea susținerii examenului de finalizare a studiilor de
licență la Facultatea de Automatică și Calculatoare, Programul de studiu
_____ din cadrul Universității Tehnice
din Cluj-Napoca, sesiunea _____ a anului universitar _____,
declar pe proprie răspundere, că această lucrare este rezultatul propriei activități
intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au
fost citate, în textul lucrării și în bibliografie.

Declar, că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au
fost folosite cu respectarea legislației române și a convențiilor internaționale privind
drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii
de examen de licență.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administra-
tive, respectiv, *anularea examenului de licență*. Sunt de acord ca, pe tot parcursul vieții,
în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu
identificat și verificat în baza datelor declarate de mine și conform copiei documentului
de identitate menționat mai sus.

Data

Nume, Prenume

Semnătura

General Advice

READ FIRST (this page should be removed from the final version):

1. If you print the thesis on paper, the three preceding pages (title page, summary page and declaration) should be printed on separate pages (one-side) and should be included in the printed paper. The summary page (the second one) must be signed by the graduate and the supervisor. The date on the declaration should be the date when the thesis is handed to the commissions' secretary.
2. On the title page, you should include the correct scientific title of your supervisor (use the Department web pages to find that).
3. Every chapter begins on a new page.
4. Do not change page margins.
5. Obey the other instructions provided in every chapter.
6. This template obeys all the formatting rules.
7. We included the `hyperref` package to generate navigation links. To produce a version for printing on paper uncomment the line containing `%\hypersetup{hidelinks}` located in the beginning of the main file `thesis_eng.tex`.

Contents

Chapter 1	Introduction	1
1.1	Context and Motivation	1
1.2	Limitations of Existing Solutions	1
1.3	The Proposed Solution: ChronoLux	2
1.4	Thesis Structure	2
Chapter 2	Project Objectives	4
2.1	Technical and Architectural Objectives	4
2.2	Metrological and Validation Objectives	4
Chapter 3	Bibliographic Research	6
3.1	Preventive Conservation and Empirical Dosimetry	6
3.2	The Metrology of Global Illumination	7
3.2.1	Rasterization vs. Ray Tracing in Scientific Simulation	7
3.2.2	The Rendering Equation and Monte Carlo Integration	7
3.2.3	Physical Material Simulation and Energy Conservation	8
3.2.4	Limitations of Offline Rendering	9
3.3	Hardware Acceleration and Heritage Digital Twins	9
3.3.1	Bounding Volume Hierarchies and Ray Traversal	9
3.3.2	DirectX Raytracing (DXR)	9
3.3.3	Variance Reduction and Texture Space Rendering	10
3.3.4	Solar Telemetry and Ephemeris Models	10
3.3.5	The Perez All-Weather Sky Model	11
Chapter 4	Analysis and Theoretical Foundation	12
4.1	High-Level Theoretical Pipeline	12
4.1.1	The Initialization Domain	12
4.1.2	The Radiometric Integration Domain	13
4.1.3	The Temporal Accumulation Domain	13
4.2	The Texture Space Ray Tracing Paradigm	13
4.2.1	The Mathematical Flaws of Screen Space Metrology	13
4.2.2	Texture Space Mapping	14
4.3	Dosimetric Path Tracing Algorithm	14
4.3.1	Stochastic Hemisphere Sampling and Energy Attenuation	14
4.3.2	The Physics of Refractive Transmission	15
4.3.3	Next Event Estimation (NEE)	15
4.3.4	Miss Evaluation and Perez Integration	16
4.4	The Parallel Determinism Problem	16
4.5	Mathematical Dose Accumulation	17
4.5.1	Temporal Integration Logic	17
4.5.2	Floating-Point Precision and Heatmap Generation	17
4.6	Conclusion	17

Chapter 5 Detailed Design and Implementation	19
5.1 Software Architecture Overview	19
5.2 Core Simulation Engine (C# CPU Orchestration)	19
5.2.1 The Chronological Engine and Time Stepping	21
5.2.2 Mathematical Astrometry and Solar Ephemeris	22
5.3 Scene Management and Hardware Interfaces	23
5.3.1 Asynchronous Runtime Mesh Parsing	23
5.3.2 Dynamic Material Serialization and Hierarchy Keying	23
5.3.3 Spatial Normalization	23
5.3.4 Deterministic BVH Building	24
5.4 Hardware-Accelerated Path Tracing (HLSL / DXR)	24
5.4.1 Memory Management and DXR Acceleration Structures	25
5.4.2 Texture Space Dispatch Paradigm	26
5.4.3 Pseudo-Random Sequence Generation and Hashing	26
5.4.4 Direct Sunlight and Stochastic Multi-Sampling Paths	27
5.5 Advanced Mathematical Models and Optimization	28
5.5.1 Perez Sky Model Atmospheric Coefficients	28
5.5.2 Hardware Traversal Mechanics (DXR and Face Culling)	28
5.6 Asynchronous Data Telemetry and UI Integration	29
5.6.1 Automated Sensor Placement	29
5.6.2 Metrological Analytics Pipeline	29
5.6.3 User Interface Toolkit and JSON Serialization Constraints	30
5.7 Deep Engine Optimizations and Memory Integrity	31
5.7.1 Ray-Origin Epsilon Offsetting and Floating Point Limits	31
5.7.2 Enforcement of Helmholtz Reciprocity and Energy Conservation	32
5.7.3 Thread Synchronization and CPU-GPU Data Dependencies	32
5.7.4 VRAM Memory Integrity and Cleanup Protocols	33
5.7.5 Data Export Payload Generation and Mathematical Variance	33
Chapter 6 Testing and Validation	35
Chapter 7 User's Manual	36
Chapter 8 Conclusions	37
Bibliography	38
Chapter 9 Appendix	40
9.1 Core HLSL Path Tracing Implementation	40
9.2 C# Orchestrator Implementations	42

Chapter 1. Introduction

1.1. Context and Motivation

The concept of cultural heritage conservation is one that is highly complex, being highly scientific and involving the balancing act between access to the public and the protection of artifacts. In museums, it is common practice to face a situation where there is a need to display old paintings, which could be as old as 16th century oil paintings, delicate pieces of cloth and even water colors without damaging them. The most ubiquitous source of degradation of such artifacts, however, is photochemical degradation caused by the presence of light.

Light Dosimetry is the metrological way of quantifying the radiative energy, measured in Lux Hours, acting upon an artifact within a specific period. Given that this damage is strictly cumulative and irreversible, it becomes imperative that preventive conservation measures be taken. For this purpose, guidelines set forth by international bodies like the International Commission on Illumination (CIE) are used to govern preventive conservation of artifacts. For highly susceptible items, the recommendation made by CIE is that the maximum limit of energy to which the artifact may be exposed should not exceed 50,000 Lux Hours annually.

1.2. Limitations of Existing Solutions

Up to now, the management of the stress induced by the light had been relying on two separate paradigms. However, both have their respective limitations when applied to today's workflows in the conservation of valuable heritage.

According to the first approach, physical light measurement can be carried out by the means of IoT sensor networks and photometers. While these devices indeed provide the essential ground truth data, they cannot help predict a threat before its occurrence; therefore, there is always going to be some delay, which makes these sensors unable to provide any proactive measure prior to a new exhibition. Besides, while measuring irradiance at a single point in space, it would take a huge amount of sensors deployed at high density in order to fully cover all possible angles of light incidence in a large object. Not only it would be expensive and unattractive, but mounting these devices on fragile pieces would simply cause irreversible damage. Finally, as every other device with a physical sensor inside, it suffers from calibration error, which needs to be compensated for by regular maintenance.

According to the second one, the issue of predicting is approached by using offline rendering engines like Radiance or Precomputed Radiance Transfer (PRT) lightmaps, which calculate the lighting distribution mathematically and in an extremely reliable way. Yet their inability to exploit hardware acceleration capabilities and being completely CPU-bound renders them impractical for real-time applications due to their poor

performance, which results in hours' worth of calculation just for a single, one-year exposure simulation done with hourly precision (which is required in order to achieve proper global illumination).

That is precisely why this offline approach does not allow any interactive exploratory process to be carried out, meaning that conservators can neither explore the possibility of introducing some interventions into the environment—like altering transmittance of window glass, changing room albedo or moving objects around—nor try some optimal combination thereof.

1.3. The Proposed Solution: ChronoLux

For mitigating the issue of the interactivity and fidelity gap, this research will introduce ChronoLux - an interactive Digital Twin platform designed specifically for near-real-time dosimetry. Making use of modern GPU acceleration capabilities through DirectX 12 Raytracing (DXR), ChronoLux will transform passive artifact three-dimensional representations into interactive, predictive, and accurate simulations.

As opposed to camera-oriented screen-space rendering through camera-to-image ray-casting performed by current generation interactive graphics platforms, ChronoLux introduces a novel architectural approach - the Texture Space Ray Tracing. Regular renderers conduct frustum culling to exclude rays that are cast beyond the view range of the camera. Such an approach is insufficient when simulating light interaction for scientific dosimetry since measurements need to be conducted persistently throughout the entire geometrical surface of an object. Instead, using the artifact's UV coordinates, ChronoLux will cast rays from those points thus baking any cumulative exposure permanently into the object's surface.

In order to guarantee accuracy and metrological precision, the simulation will run under the Perez All-Weather Sky Model that will provide exact positioning of sunlight along with the skylight irradiance. The core of the algorithm will use the stochastic Next Event Estimation (NEE) Monte Carlo path tracer that will allow sampling separate components of illumination such as direct solar beams and diffuse scatterings independently. Overall, this approach provides curators with a non-destructive sandbox environment that enables instant dosimetric measurements without any drop in performance at approximately 60 fps rate.

1.4. Thesis Structure

Subsequently, the body of this paper is arranged in such a way that all the necessary information regarding ChronoLux will be delivered, from theory to experimental confirmation.

In Chapter 2, the main goals of this study will be described, namely, both interactive goals and metrological ones. Chapter 3 represents an extended review of the bibliography and describes recent achievements in the areas of physical dosimetry, offline ray tracing, and hardware accelerated global illumination. Chapter 4 contains the analysis of the theory of light transport, including the explanation of Lambert's Cosine Law, and Monte Carlo integrals. Chapter 5 describes the process of creation of the Digital Twin and focuses on the Texture Space Ray Tracing algorithm, as well as Next Event Estimation kernel. In Chapter 6, the experimental validation and testing of the proposed

framework is described in detail, including different conservation case studies and proving the mathematical consistency of the Monte Carlo convergence. In addition, Chapter 7 presents an elaborate user guide to using the software, and in Chapter 8 - conclusions and future perspectives are stated.

Chapter 2. Project Objectives

The main focus of this thesis is to create a hardware-accelerated Digital Twin, called *ChronoLux*, which will provide an interactive and physically-based lighting simulation for the conservation of cultural heritage. This solution is designed to fill the important missing link between computationally-heavy offline rendering systems (such as Radiance), which can be proven mathematically, and real-time IoT sensing devices.

In order to achieve this high-level goal and provide the necessary combination of interactivity and scientific accuracy, the development process is carried out based on certain technical and metrological goals. The high-level architecture related to these goals is outlined in Figure 2.1.

2.1. Technical and Architectural Objectives

- 01. Texture Space Ray Tracing Integration:** To address the drawbacks associated with traditional screen space algorithms, such as frustum culling that results in critical out-of-screen lighting information being removed from calculations, a special compute shader for DirectX 12 Raytracing (DXR) would have to be used to launch rays directly from the unwrapped UV texture map of 3D models in order to permanently bake the cumulative radiometric energy into all of its geometric surface regardless of the location of the observer.
- 02. Physical Environmental Modeling:** In order to combine the Perez All-Weather Sky Model with a special Next Event Estimation (NEE) compute algorithm to create a scientifically rigorous method to decouple the direct beams of sunlight from the diffuse ambient lighting, thus generating physically correct global illumination without loss of real-time processing ability.
- 03. Asynchronous Compute Architecture:** For a proper separation of concerns between the user interface layer and computationally demanding GPU path tracing engine, the program should allow users to modify physical properties (transmission coefficients of window glazing or wall reflectance, for instance) via an interactive UXML/USS user interface without breaking the continuity of the 60 FPS viewport interaction.

2.2. Metrological and Validation Objectives

- 04. Headless Statistical Telemetry:** In this study, an automated approach has been developed for virtual point-probe Lux sensors which operate purely in the background. The sensors are created for the purpose of computing spatial variance (σ^2), maximum exposure dose, and standard Monte Carlo convergence error boundaries while not interfering with the primary rendering process.

O5. Empirical Scenario Validation: Explicit validation of the above approach is achieved by means of running simulations of an entire year of meteorology (i.e., 8,760 hours of continual processing) for various preventive conservation scenarios. It is essential for establishing the efficacy of dynamic environmental control actions like the application of UV-absorbent windows or optimizing geometric occlusion of susceptible artworks.



Figure 2.1: High-Level Concept Architecture of the ChronoLux framework, illustrating the pipeline from raw heritage asset ingestion to real-time dosimetric data export.

The subsequent chapters of this thesis will methodically detail the design, theoretical foundation, programmatic implementation, and scientific validation of these stated objectives.

Chapter 3. Bibliographic Research

The creation of a hardware-accelerated Digital Twin for online light dosimetry is the result of the intersection of several complex scientific fields. The following section will provide an in-depth analysis of the literature related to preventive conservation metrology, mathematics of global illumination, and modern graphics processing technology. By analyzing the various approaches used today, the theoretical basis of this thesis will be defined, together with the limitations of current methods to be overcome by *ChronoLux*.

3.1. Preventive Conservation and Empirical Dosimetry

The need for light dosimetry is necessitated by the photochemically reactive nature of heritage objects. According to Michalski [1], both visible light and ultraviolet radiation are responsible for causing permanent molecular decay in natural and synthetic materials. In terms of photon energy, there is an inverse relationship between wavelength and energy level. The quantum energy of ultraviolet light, which occurs at wavelengths between 10 and 400 nm, is enough to break covalent bonds between pigment molecules, binding agents, and structural fibers.

Since light-induced deterioration is entirely accumulative, healing or reversing any damage is impossible simply by putting the artwork in darkness because the bonds have already been broken at a molecular level. To mitigate such problems effectively, the CIE set rigorous conservation limits and classified materials based on their varying degrees of vulnerability to light [2].

- **Insensitive Materials:** Artifacts are sensitive to exposure to light to differing extents. Materials like metals, stone, and ceramics are not vulnerable to photodegradation and, hence, have no special lighting requirements.
- **Low Sensitivity Materials:** Oil and tempera paintings, frescoes, and wood used in structures are capable of tolerating moderate exposure, up to 600,000 Lux-Hours per year.
- **High Sensitivity Materials:** Water colors, pastels, tapestries, and old papers, on the other hand, are extremely delicate and can only tolerate 50,000 Lux-Hours exposure in a year.
- **Extreme Sensitivity Materials:** Silk, certain unstable dyes, and early photos must be protected against all dangers posed by excess light, which is usually restricted to 15,000 Lux-Hours annually.

Going beyond these levels will hasten the process of entropy of artifacts, making it mandatory to establish accurate forecasting processes.

Traditionally, the approach that has been used extensively for evaluating such exposure is the measurement of incident light intensity. According to Silva et al. [3], the use of affordable data loggers to measure the microclimate in these locations is emphasized. In addition to this, Chianese et al. [4] describe the concept of “Internet of Cultural

Things” by suggesting the presence of dense IoT-based sensors within museums.

Discussion: While empirical sensor networks provide irrefutable real-world data, there exist several operational constraints associated with such systems that significantly limit their potential for use in modern conservation practices. First, it should be noted that these devices are purely reactive, with a physical sensor sending a signal when light interaction occurs, thus not enabling any preventive action before an exhibit is opened to light exposure. Second, the accurate simulation of the spatial distribution of irradiance at a huge three-dimensional surface would require installing an unrealistic amount of physical sensors on the subject. For instance, using thousands of sensors to cover a Renaissance sculpture would be both costly and visually unappealing. It could also be dangerous due to the fragility of such objects. As a result, to shift from reaction to prevention and simulation, mathematical models are required.

3.2. The Metrology of Global Illumination

3.2.1. Rasterization vs. Ray Tracing in Scientific Simulation

The visualization of virtual worlds in computer graphics has traditionally been implemented using rasterization algorithms. Rasterization maps 3D polygons to the 2D surface of the screen, computing their visibility through depth buffering. While extremely fast and highly suited for interactive purposes like games, rasterization is essentially incapable of scientific dosimetry. As the algorithm only takes into account direct visibility of objects (frustum culling), it heavily relies on ad hoc methods to create indirect lighting, including Screen Space Ambient Occlusion (SSAO), shadow mapping, and baked light probes, introducing significant bias.

Ray tracing, on the other hand, explicitly traces the actual path taken by the photons as they emanate from the light source, reflect from the geometry based on its material properties, and refract if necessary. Traditionally used for producing photorealistic images, ray tracing is the de facto standard for creating visual effects (VFX) for films and, most recently, top-tier video games due to the ability to trace perfectly accurate mirror reflections, soft shadows, and transparent materials’ refractions. In both of the cases, however, the goal remains that of deceiving human eyes and presenting realism without paying too much attention to the physical aspects in order to reduce rendering time.

Conversely, in order to achieve scientifically accurate dosimetry, one would need to simulate purely physical behavior of light with no regard to any virtual cameras or human perception involved. For that purpose, simulations need to be performed using unbiased and purely mathematical ray tracing which will allow the traced photons to serve as purely radiometric data. This technique is mathematically provable and thus represents the only possible algorithm for the measurement of the actual irradiation and Lux-Hour accumulation in a virtual world.

3.2.2. The Rendering Equation and Monte Carlo Integration

The key model that forms the backbone of physically based ray tracing is known as the Rendering Equation and was first introduced by Kajiya [5]. It represents a balance between the amount of radiance leaving a surface point through all directions, and the

summation of two quantities – radiance coming from emitting objects and scattered radiance from the hemisphere around the surface point.

Due to the fact that solving this infinite-dimensional equation is impossible for any graphics engine, an approximate method called Monte Carlo integration must be used. It entails sampling a random subset of rays in order to estimate the outcome of the integral. In other words, the more samples used, the higher the precision of the estimate – a principle defined by the Law of Large Numbers.

However, uniformly sampling random rays proves to be rather wasteful because many of them will not interact with any source of illumination, hence producing a noisy picture with high variance. In order to overcome this issue, the importance sampling technique can be applied. This approach uses a probability density function (PDF) that defines where rays should be cast based on the expected light distribution (e.g., cosine-weighted importance sampling for Lambertian surfaces).

In case the environment is enclosed and bounded, stochastic termination methods, like Russian Roulette, need to be applied in order to avoid infinite bouncing of rays. Instead of limiting their path length with an artificially chosen number of bounces, Russian Roulette ends the path when encountering dark surfaces which absorb more light.

3.2.3. Physical Material Simulation and Energy Conservation

Fidelity of the illumination is based on the rigour of material definitions. When a photon hits a geometrical surface, the physics behind what happens next can be described by the surface's Bidirectional Reflectance Distribution Function (BRDF) or Bidirectional Transmittance Distribution Function (BTDF). The combined effect describes the distribution function for the way the irradiance is reflected, absorbed, and transmitted through the hemisphere defined by the surface normal.

Modern computer graphics often use elaborate PBR materials that can be described by the state-of-the-art microfacet models such as the GGX distribution. According to Pharr et al. [6], these microfacet equations are employed to simulate the fine details of surface roughness and metals to achieve photorealistic visuals. However, their complex nature comes at an enormous computational cost while making a relatively minor contribution to the estimation of radiometric energy density on flat architectural surfaces.

ChronoLux does not use complex visuals of modern PBR materials but focuses on thermodynamically rigorous calculations. In metrological terms, according to the First Law of Thermodynamics (Energy Conservation), the sum of Reflectance (R), Transmittance (T), and Absorptance (A) of any material must be equal to one ($R + T + A = 1$). Thus, the reflectance of any hypothetical material reflecting 60% and transmitting 30% of the incident irradiation, like a tinted window glass, must have 10% absorptance.

Correctly accounting for these factors is critical for preventive conservation purposes. Walls are normally Lambertian, which means the photon is equally distributed in all directions after interaction with the wall surface, affecting the level of indirect illumination in the room. By contrast, windows imply complex interactions of photons and materials including transmission and refraction. A correct estimation of the transmittance graph of the UV-blocking glass or reflectance factor of polished marble flooring is crucial as it determines how the Lux-Hour dosage will accumulate on the artifacts.

3.2.4. Limitations of Offline Rendering

The foremost example of these approaches in architecture and conservation is the Radiance program [7]. The Radiance approach combines backwards ray tracing together with irradiance caching to solve the Rendering Equation with extremely high metrological precision.

Discussion: In contrast to other approaches to rendering, Radiance allows for the creation of lighting distributions that are provably correct from a mathematical standpoint. For this reason, it has become the gold standard for offline dosimetry calculations. The reliance on CPU bound processing however leads to significant inefficiencies. It takes several hours for the calculation of a full year of exposures, thus ruling out any interactive examination of the potential conservation strategies.

One of the alternatives to this method, Precomputed Radiance Transfer (PRT) [8], aims at solving the problem of real-time global illumination. PRT is achieved through precomputation of light transport into spherical harmonics. However, its applicability suffers greatly when the environmental effects are not static. An adjustment of the object orientation or transmittance properties of window glazing requires recalculating the entire PRT lightmap.

3.3. Hardware Acceleration and Heritage Digital Twins

The creation of highly accurate dosimetry with the capability of real-time interaction requires architectural thinking to be updated. As stated by Jouan & Hallot [9] and Niccolucci et al. [10], the concept of "Heritage Digital Twin" goes beyond that of traditional 3D representation or, otherwise known as HBIM, and becomes a dynamic replication continuously fed with data from reality to make decisions about preventive conservation.

3.3.1. Bounding Volume Hierarchies and Ray Traversal

The introduction of the Rendering Equation in this context at 60 Hz renders a need for complex spatial acceleration structures. Finding an intersection of a given ray with several million raw geometric triangles mathematically becomes impossible to compute within seconds. For that reason, scenes have to be split into Bounding Volume Hierarchy (BVH).

BVH is a kind of tree-like structuring where all geometries are placed into Axis-Aligned Bounding Boxes (AABB). The root of such structure contains the whole scene and, consequently, all geometries are successively divided by nodes until leafs, containing actual triangles, are reached. When tracing rays, we are checking intersections with bounding boxes. In case a ray misses a parent AABB, the algorithm ignores all of those millions of triangles turning an $\mathcal{O}(n)$ problem into a logarithmic $\mathcal{O}(\log n)$ search.

Surface Area Heuristi (SAH) is utilized to construct these trees optimally such that it reduces the probability of any intersection between rays and bounding boxes by assessing the cost of node traversal and triangle intersection.

3.3.2. DirectX Raytracing (DXR)

Traditionally, BVH traversal and construction was carried out using software on CPUs. In the words of Burgess [11], this approach was turned upside down in terms

of the NVIDIA Turing GPU architecture that came equipped with hardware silicon RT Cores for accelerating BVH traversal and triangle intersection.

The DXR API provides this functionality to developers by offering two acceleration structures for managing static and dynamic objects respectively. The first of these is the Bottom Level Acceleration Structure (BLAS). The bottom level structure contains all the raw triangle information of each mesh. Although extremely optimized, its creation requires high computational effort due to the complexity involved in the process. In contrast, the second acceleration structure—the Top Level Acceleration Structure (TLAS)—contains light instance references to BLAS that allow transformations to be applied instantly without rebuilding.

DXR provides an updated shader pipeline that makes use of a wide array of frameworks like ChronoLux:

- **RayGeneration Shader:** The first shader program used in the pipeline, responsible for generating primary rays. Unlike the traditional method of launching rays via camera, this process launches rays in Texture Space.
- **ClosestHit Shader:** Triggered upon successful intersection between a ray and solid geometry. At this point, it evaluates properties like BRDF, physics albedo, and diffuse interreflection.
- **AnyHit Shader:** Executed before Closest Hit to evaluate opacity or perform alpha test. This shader is required in scientific dosimetry to accurately calculate physics transmissivity across refractive window glazing without stopping the shadow ray.
- **Miss Shader:** Executed once the ray travels past local geometry. When working on environmental simulations, Miss shaders evaluate global illumination techniques such as Perez All-Weather Sky Model [12] to accurately compute solar irradiance.

3.3.3. Variance Reduction and Texture Space Rendering

For optimal performance of Monte Carlo Integration, Veach and Guibas [13] provided the necessary mathematical foundations for incorporating sampling algorithms that enabled the utilization of Next Event Estimation (NEE). NEE greatly helps to reduce the problem of variance through explicit casting of a shadow ray to the main light source in each bounce. The technique ensures fast convergence, even when there is indirect lighting, since it disassociates sunlight rays from ambient ones.

Moreover, for isolating fast ray traversal from the boundaries of the viewport of a camera, Munkberg et al. [14] presented modern texture-space cache methods. Traditional screen space renderers throw away useful lighting information that falls out of the current frustum view. The new technique renders images in texture space by shading objects in their corresponding UV mappings regardless of camera occlusions.

3.3.4. Solar Telemetry and Ephemeris Models

A precise simulation, especially one involving an entire meteorological year consisting of 8,760 hours, demands the exact position of the Sun in the sky. The solar position determines the angle of incidence of radiation, thus the shadows cast on any objects, and the amount of transmitted light through windows.

This can be obtained using various mathematical models of solar position from the scientific computing libraries, known as Solar Ephemeris. For instance, there exists a library called Solar Position Algorithm (SPA) formulated by Reda and Andreas [15]. This library calculates the solar zenith and azimuth positions based on latitude and longitude

position of the heritage sites, together with the Julian day and local time at any given point.

Apart from precision, calculation of the solar position is vital for computational optimization. Path tracing is quite computational intensive. Calculation of solar zenith position prior to the execution of the path tracing process allows the simulation to know if the Sun is below the horizon line. In case when it is night time (time between sunset and sunrise), then the process of path-tracing for the specific hour is not necessary, therefore reducing computation time significantly.

3.3.5. The Perez All-Weather Sky Model

The correctness of the virtual environment dose estimation model depends on the accuracy of the external light source driving it. In traditional three-dimensional rendering, the simplest form of light direction is usually used to simulate sunlight and set the sky color to uniform color; however, the physical process of atmospheric light scattering has some significant complexity. The complexity is solved by the use of scientific metrology through precise atmospheric math calculations, namely, the Perez all-weather sky model [12].

The Perez model is the empirical mathematical model that calculates the relative luminance of any sky point with regard to real meteorology conditions. Opposed to other simple models describing only ideal conditions of absolutely sunny or cloudy weather, Perez uses two atmospheric parameters:

- **Clearness (ϵ):** An index used to describe the ratio between direct solar radiation and diffuse sky radiation, giving a measure of how much clouds there are and their thickness.
- **Brightness (Δ):** An index that measures atmospheric luminous opacity, dependent on aerosol concentration and humidity.

With all these parameters taken into account along with precise solar angle and azimuth values, the model creates a very precise continuous luminance distribution. Such an approach incorporates modeling of two important scattering effects within the atmosphere, namely, circumsolar brightness, which results from Mie scattering near the sun and horizon brightness which is a result of the luminance gradient formed due to Rayleigh scattering in denser atmospheric layers around the horizon. In terms of the ChronoLux system, once the path traced ray leaves the building using DXR Miss shader, its vector direction is used to obtain a physical spectral radiance value for this particular micro angle within the sky dome.

Conclusion: The approaches examined within the current chapter demonstrate that even though offline rendering tools such as Radiance give precise results, and IoT sensors provide actual information, these methods cannot predict real-time changes and thus require interaction. Through a combination of the precise calculations of Monte Carlo path tracing algorithm and NEE technique suggested by Veach [13] along with the Perez sky model [12] and powerful hardware acceleration obtained through use of DXR [11] and BVH traversal along with texture space ray tracing [14], the proposed approach will fill this gap in preventive conservation.

Chapter 4. Analysis and Theoretical Foundation

In this chapter, a clear definition of theoretical algorithms, mathematical abstractions, and overall logic behind the *ChronoLux* design is attempted. Following the literature review, it becomes apparent that current methods of simulation suffer from significant shortcomings that render them unsuitable for application within interactive heritage metrology. Subsequently, in the following section, the theoretical measures suggested to tackle the aforementioned problems are outlined, focusing purely on abstract data manipulation and physics of light transfer.

4.1. High-Level Theoretical Pipeline

The *ChronoLux* pipeline is an endless sequence of mathematical computations that turns abstract three-dimensional data into radiometric energy build-up. From an architectural perspective, the *ChronoLux* system can be broken down into three separate domains. These include the Initialization Domain, Radiometric Integration Domain, and Temporal Accumulation Domain. As described earlier in Figure 2.1, *ChronoLux* operates on a closed-loop basis, using time-discrete iterations for its operations.

4.1.1. The Initialization Domain

Before the execution of radiometry calculations, a physical definition of the virtual environment is required. This initialization starts from the import of geometrical mesh information. From the theoretical standpoint, each artifact consists of a discrete collection of interconnected triangles forming a manifold geometry. Each of these triangles is connected to physically realistic material behavior, described by the strictly conservative thermodynamic equation ($R + T + A = 1.0$) outlined in Chapter 3.

The geographical and temporal status of the simulation is established at the same time. According to the theoretical description, the coordinates of the geographic latitude (Φ), geographic longitude (λ), geographic timezone offset, and the initial Julian Day of the physical heritage location are necessary inputs into the astronomy ephemeris calculation algorithm. After the completion of the first stage, a Bounding Volume Hierarchy (BVH) with a statistical optimization applied to the spatial information, as well as the solar zenith angle (θ_z), and solar azimuth angle (γ_s) for the current temporal step (t) are derived. If the calculated solar zenith angle is less than zero ($\theta_z < 0$), then, due to the system's automatic culling mechanism, the radiometric calculation process does not have to be executed at all, moving to the next temporal iteration point.

4.1.2. The Radiometric Integration Domain

After both sets of parameters have been initialized, the system proceeds to the next stage, which is referred to as the Radiometric Integration Domain. This is where the primary mathematical algorithm for calculating irradiance takes place. Unlike in conventional rendering techniques, which use the projection of the view frustum onto the two-dimensional space, all computations take place in Texture Space, an idea that will be further explained in Section 4.2.

In this domain, a computational solution is sought for the Rendering Equation at all acceptable points of the artifact's surface. Rays are propagated stochastically through the hemisphere to calculate both direct sunlight and diffuse ambient light using the technique of Next Event Estimation (NEE). Directions of escaping rays are compared to the Perez All-Weather Sky Model, allowing the system to receive exact spectral values of radiance. At the end of computations, a two-dimensional grid of data that corresponds to instantaneous physical irradiance (in Lux) is obtained.

4.1.3. The Temporal Accumulation Domain

The last theoretical step comprises the constant temporal summation of the irradiance values at each moment of time. Photochemical decomposition is always an accumulative process, so the calculated radiometric information will be reused rather than thrown out after one-time processing. In particular, the instantaneous illuminance (Lux) value is multiplied by the temporal duration of the current simulation step (Δt) to derive an accumulative dosage in Lux-Hours. The accumulative dosages will be added up using a persistent floating point array. After that, the temporal counter ($t = t + \Delta t$) is advanced, and the procedure is iterated until the end of the chosen period of time (e.g., meteorological year).

4.2. The Texture Space Ray Tracing Paradigm

The first major theory in which *ChronoLux* deviates from regular computer graphics is that of using the Texture Space Ray Tracing method of computation. To illustrate why such a computational method is needed in science metrology, its advantages, as well as the mathematical limitations of conventional methods, will be demonstrated below.

4.2.1. The Mathematical Flaws of Screen Space Metrology

First, in a three-dimensional engine, there is always a virtual viewer defined by a set of parameters including a field of view, distance from the screen, and near/far planes. During rendering, a two-dimensional pixel grid is used. For each pixel, there is a ray traced from the camera's optical point through the pixel into three-dimensional space.

Such an approach is mathematically exact, but it is flawed when we need to accumulate some radiation on all geometric surfaces. First, the problem is related to occlusion and perspective projection. When tracing rays from the virtual viewer, it becomes clear that the system will trace those rays that actually intersect the polygons. Any surface turned away from the viewer will not be computed because it simply will not be covered by any rays emanating from the viewer. It is obvious that if a statue is turned around the wrong way, no radiation can be accumulated on its posterior surface; the same holds true for other surfaces.

4.2.2. Texture Space Mapping

To accomplish a full separation between the simulation and the artificial restrictions of camera placement, *ChronoLux* uses Texture Space Mapping. In 3D geometry, each vertex of the polygon corresponds to a point on the 2D UV plane through (u, v) coordinates in three-dimensional geometry, which is called the UV map.

The process of traversing the UV map in the *ChronoLux* theory does not involve traversing the pixels on the computer screen but involves traversing the discrete texels on the UV map. This allows for the recreation of geometry in world space by calculating P (position) and N (surface normal).

Let the UV map be a 2D matrix M of size $W \times H$. For every element $M_{i,j}$:

1. Read the pre-computed world-space position vector $P = (P_x, P_y, P_z)$ corresponding to the (u, v) coordinate.
2. Read the pre-computed world-space normal vector $N = (N_x, N_y, N_z)$.
3. Use P as the origin point for the dosimetric path tracer.
4. Use N to orient the sampling hemisphere.

This method ensures that each infinitesimally small patch of the surface of the object is evaluated simultaneously, without regard to the virtual camera orientation. Global consistency of the physical light transport throughout the entire structure of the heritage object is guaranteed.

4.3. Dosimetric Path Tracing Algorithm

For the sake of simplicity, the origin (P) and the direction (N) of the rays as established using the Texture Space technique are known; hence, it is necessary to solve the global illumination equation in order to find the exact amount of energy hitting the particular point. This task can be accomplished using a highly efficient Monte Carlo algorithm with Next Event Estimation (NEE).

4.3.1. Stochastic Hemisphere Sampling and Energy Attenuation

Firstly, to find the total irradiance on a surface, the sampling directions have to be found, which involves generating directions distributed evenly over the upper hemisphere centered at the point where the sampling takes place. Within the framework of the Monte Carlo algorithm, such distribution of samples can be represented using a stochastic directional vector ω_i .

To improve convergence rates and minimize variance, *ChronoLux* uses Cosine-Weighted Importance Sampling. Instead of uniformly sampling random vectors, the probability density function used here tilts towards the vertical axis that passes through the surface normal. This method mirrors the physics behind lambertian diffuse surfaces when light coming from the vicinity of the surface normal carries more energy (proportional to $\cos(\theta)$) than that coming in from a large angle θ .

As these stochastic rays traverse the digital scene, they interact with other surfaces. The core principle behind this interaction is energy absorption, meaning the photonic path will dissipate its energy after each encounter with another surface. Thus, an object placed in a room with dark mahogany walls (with high Absorptance, A) will reflect indirect rays with significantly less energy than those reflected from white plaster (high Reflectance, R).

This effect can be easily modelled mathematically through the use of a throughput multiplier. Starting with an initial value of 1.0, which means perfect energy transmission, throughput gets multiplied by Reflectance R at each encounter. As $R < 1.0$ for any physical surface (following the law of conservation of energy $R + T + A = 1.0$), the ray loses energy at an exponential rate after each reflection.

In order to prevent simulation from becoming infinitely long, because of an endless number of bounces in enclosed space, a finite bounce depth limit should be set. For example, typical indirect path lengths do not exceed 3 or 4 bounces since additional reflections carry too little residual energy to matter further.

4.3.2. The Physics of Refractive Transmission

A basic element of heritage metrology involves evaluating the impact of external light that passes through physical obstacles, especially the effects of window glazing. This requires realistic modeling of refraction for light.

When a path traced ray meets a boundary between two media (say, a pane of glass), the ray does not necessarily stop or bounce off. Rather, it changes direction based on the IORs of the two media (air-glass) according to Snell's Law:

$$\frac{\sin \theta_1}{\sin \theta_2} = \frac{n_2}{n_1} \quad (4.1)$$

In the above-mentioned framework, n_1 and n_2 stand for the refractive indices of the media from which the ray starts and where it reaches, respectively; while θ_1 and θ_2 stand for the angles of incidence and refraction, respectively, regarding the normal to the interface.

As far as the *ChronoLux* abstract model is concerned, the transmission vector is calculated via Snell's Law whenever the beam penetrates a transparent body, allowing the ray to proceed with its travel through the solid glass. At the same time, the throughput of the ray is adjusted with respect to the particular material's Transmittance (T) factor. If a conservator applies UV-blocking film with a Transmittance of 0.45, the ray throughput is immediately decreased by 55%, thus providing an adequate simulation of the real-world effect produced by the intervention.

4.3.3. Next Event Estimation (NEE)

In a pure Monte Carlo path tracer, all sampling is random, and the tracing algorithm can use only random reflection from surfaces. The chance of finding the sun's small angular diameter through random reflections inside a dark museum is practically nil; therefore, there is significant variance in the simulation, and one would need to calculate millions of samples to converge toward the correct answer, rendering the calculation unfeasible at least with respect to the 8,760-hour simulation.

To alleviate this problem, *ChronoLux* employs the technique known as Next Event Estimation (NEE).

At every single interaction point P across the entire bounce sequence, the algorithm performs the following logic:

1. **Direct Shadow Ray:** A deterministic vector ω_{sun} is generated with the direction towards the exact current location of the sun. "Shadow ray" tracing occurs along ω_{sun} . If this ray leaves the bounded region without intersection with any opaque objects, the direct irradiance is calculated using the Perez model and added to the total dose. In case there is an intersection with a transparent object (such as

window glass), the light intensity will be reduced by a factor of T which is obtained using Snell’s law.

2. **Indirect Bounce Ray:** At the same time, a stochastic vector ω_i is generated using Cosine-Weighted Importance Sampling. Tracing is performed along this vector to calculate the indirect light caused by reflection from walls and floors.

Through its clear disentanglement of the deterministic solar irradiance and the random scattering of the ambient sky dome, NEE guarantees that the massive energy content of the sun is properly accounted for at every stage of the simulation, thus reducing the number of samples needed by many orders of magnitude and ensuring metrological stability. The total irradiance of the starting surface point is thus computed as the sum of all successful shadow rays cast from the initial surface point through multi-bounces.

4.3.4. Miss Evaluation and Perez Integration

The last piece of the integration domain puzzle is that of Miss evaluation. When the stochastic ray (be it the direct shadow ray or the indirect bounced ray) finally escapes the digital twin architecture, it will need to retrieve the radiance value from the physical world surrounding the building.

Contrary to returning a hardcoded color, the escaping ray direction, ω_{escape} , is mathematically inserted into the Perez All-Weather Sky Model function. This function calculates the elevation (θ_z) and azimuth (γ_s) of ω_{escape} against the current solar location (θ_z, γ_s) and uses the meteorological data of Clearness (ϵ) and Brightness (Δ) to calculate the physical luminance value of ω_{escape} . If ω_{escape} happens to be near the solar disk, the result will undergo an exponential increase to simulate the Mie circumsolar scattering effect. Otherwise, if ω_{escape} is close to the horizon, then the Perez output transitions smoothly to simulate Rayleigh scattering.

4.4. The Parallel Determinism Problem

One of the problems that arises when moving from the abstract realm of mathematics into practical implementations in highly parallel architectures, such as today’s GPUs, is non-determinism in floating-point additions.

A typical parallel tracing algorithm traces hundreds of millions of rays independently using thousands of threads simultaneously. Once the rays finish traversing the scene, all threads attempt to write their irradiance into a shared matrix of accumulations M .

As opposed to integer numbers, floating-point operations are non-associative, meaning that $(A+B)+C \neq A+(B+C)$ because of tiny differences that occur during precision rounds in IEEE 754. As a result, the final value of luminance will be different each time the algorithm runs if the order of writing to the accumulation matrix M by the threads is random.

In video games, this minute difference is not detectable. When applied to science, however, non-determinism becomes unacceptable since an analysis tool must return the same results every time a test simulation is conducted.

The solution to the problem for the *ChronoLux* framework involves making sure that ray tracing is done and accumulations happen in a predetermined manner, through mathematical structuring of geometry or seed points. This solution may add minimal computational expense, but it guarantees mathematical consistency and reproducibility.

4.5. Mathematical Dose Accumulation

The key theoretical objective of the *ChronoLux* tool is not only creating a photographically accurate representation of light but rather finding the overall photochemical risk posed over a long period of time. It implies moving from considering an instantaneous value of illumination to a cumulative dosimetry one.

4.5.1. Temporal Integration Logic

Let $E_t(P)$ be the instantaneous physical irradiance (measured in Lux) estimated with the help of dosimetric path tracer on a particular surface point P in a particular moment of time t .

Light illumination in a physical museum is a continuous process, while a digital simulation has to discretize time into particular segments (Δt). The choice of Δt significantly affects not only the accuracy of the calculation but also its performance. A traditional meteorological simulation usually takes 1 hour for each Δt . It allows tracking a motion of the Sun and cloud cover change in 8,760 steps within a typical year.

Cumulative dose $D(P)$ delivered to a particular surface point P for the whole period of time T is theoretically equal to the following sum:

$$D(P) = \sum_{t=0}^T E_t(P) \cdot \Delta t \quad (4.2)$$

Where:

- $D(P)$ is the total accumulated exposure in Lux-Hours.
- $E_t(P)$ is the calculated irradiance in Lux at time step t .
- Δt is the duration of the time step in hours.

4.5.2. Floating-Point Precision and Heatmap Generation

In order to retain the accumulating radiation dosage $D(P)$, the program uses the previously discussed Texture Space M matrix. Standard bitmap file formats (such as the 8-bit PNG or JPEG) cannot be used to store dosimetric data, as they can only store values from the set of integers ranging from 0 to 255. An extremely sensitive piece of art may amass as much as 50,000 Lux-Hours during its lifetime, which will quickly surpass the capacity of the 8-bit variable.

Therefore, according to the theory, the system makes use of a 32-bit floating point data structure (RFloat). A 32-bit float can theoretically encode 3.4×10^{38} , providing practically infinite precision for unbounded accumulation.

As the simulation advances within the temporal dimension, the matrix elements continuously increase in value. The abstract mathematical information is then mapped to a heatmap for visual analytical representation of areas where photosensitivity risks are critical.

4.6. Conclusion

Thus, in conclusion, the theories described in the current chapter make *ChronoLux* a purely metrological simulation software. Abandoning the bias-laden approach of the Screen Space rendering in favor of the Texture Space ray tracing method guarantees

complete geometrical coverage. Furthermore, Next Event Estimation, together with the Perez Sky Model algorithm, ensures exact computation of the light transport. Finally, accumulating irradiation in an arbitrarily precise floating point matrix allows for accurate simulation of photochemical degradation for meteorological years.

Chapter 5. Detailed Design and Implementation

This chapter provides an exhaustive breakdown of the software engineering architecture, the hardware-accelerated algorithms, and the rigorous mathematical systems implemented to realize the *ChronoLux* digital twin. Transitioning from theoretical radiometry and conservation physics into a highly performant desktop application requires a sophisticated orchestration of CPU scheduling, unmanaged memory transfers, and massive GPU parallelization.

Predictive analysis workflows governed by the International Commission on Illumination (CIE) and Michalski guidelines for preventive conservation historically suffer from a lack of interactivity. Traditional offline software like Radiance demands hours of CPU-bound calculations for a single temporal simulation. *ChronoLux* resolves this bottleneck by implementing a hybrid architecture utilizing the Unity 3D engine and the DirectX 12 graphics API to unlock hardware-accelerated DirectX Raytracing (DXR). This chapter systematically dissects each component of this pipeline, from mathematical astrometry to GPU memory management and asynchronous telemetry extraction.

5.1. Software Architecture Overview

To process the massive volume of ray queries required for an annual dosimetric simulation without halting the operating system, a strictly decoupled architecture is employed. The software stack is bifurcated into two distinct domains: the C# CPU layer handling user interface, file I/O, deterministic structural sorting, and astrometric mathematics; and the HLSL Compute Shader layer executing the heavily parallelized Monte Carlo integrations directly on the GPU.

As seen in Figures 5.1 and 5.2, the flow of data is strictly managed. Project parameters are serialized to JSON via the `ChronoProject` object, parsed by the `ProjectManager`, and fed into the `LightDoseSimulator`. This orchestrator acts as the simulation's beating heart, stepping time forward and delegating ray dispatching to the DXR kernels.

5.2. Core Simulation Engine (C# CPU Orchestration)

The core logic of the application must ensure mathematical determinism, prevent hardware starvation, and respond dynamically to user inputs.

As shown in Figure 5.3, the `LightDoseSimulator` depends on several critical subsystems: `SunCalculator` for astrometric mathematics, `IrradianceBaker` for hardware dispatching, and `RuntimeModelLoader` for asynchronous file I/O.

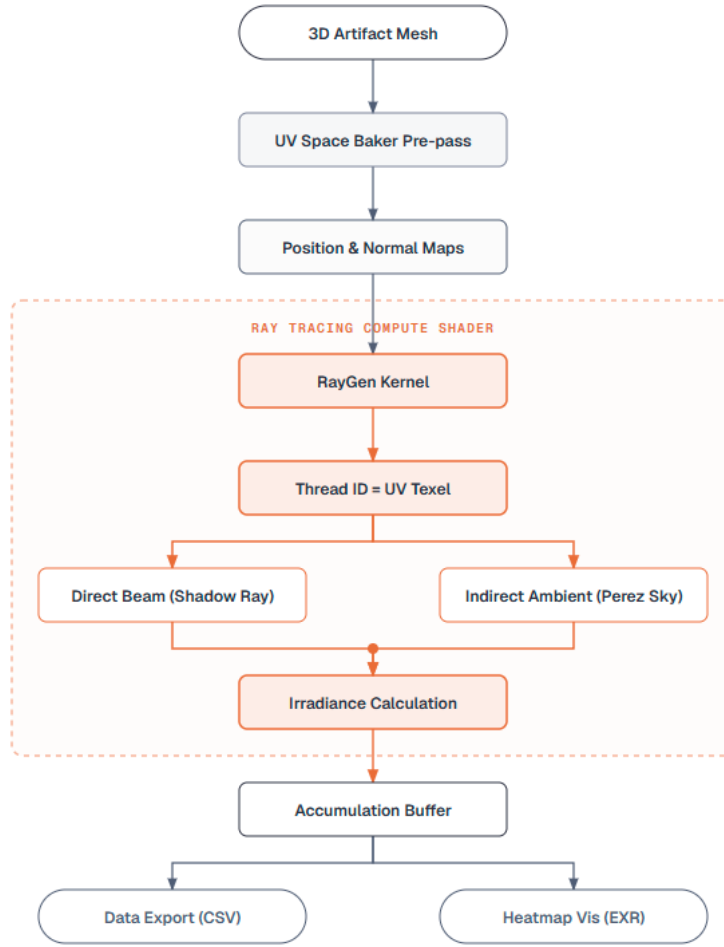


Figure 5.1: High-level simulation pipeline demonstrating the decoupling of CPU orchestration from GPU execution.

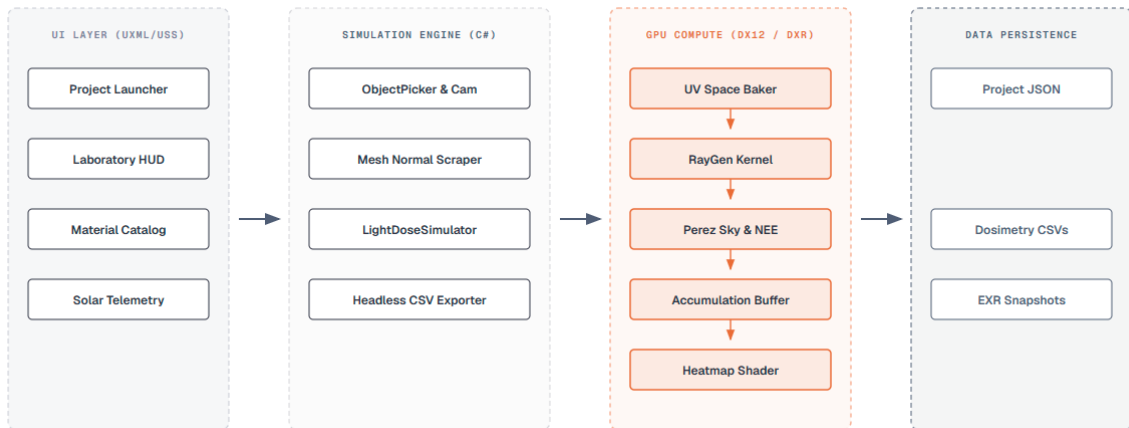


Figure 5.2: Overall Architectural System design of *ChronoLux*, detailing data paths between the Engine, the GUI, and the OS filesystem.

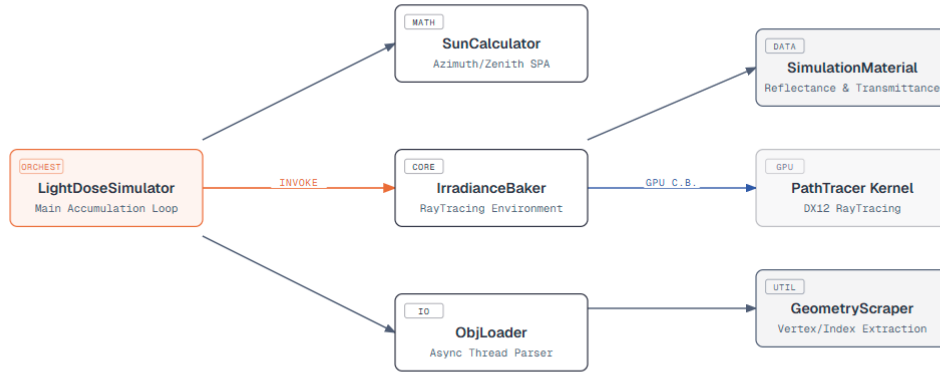


Figure 5.3: C# CPU Class Architecture and Data Flow orchestration within *ChronoLux*.

5.2.1. The Chronological Engine and Time Stepping

The primary responsibility of the `LightDoseSimulator` is to advance the master clock Δt and generate the precise solar altitude and azimuth required for the Perez sky model.

The simulation logic must process an entire meteorological year while preventing the Operating System from triggering a Timeout Detection and Recovery (TDR) crash, which forcefully restarts the graphics driver if a single task takes longer than approximately 2 seconds. This is achieved via a C# `IEnumerator` Coroutine (`RunSimulationInternal`). As detailed in Appendix 9.2 (Listing 9.3), the time-stepping loop is entirely dynamic. It processes the user-defined `startDay` and `endDay` linearly.

For every single day, the Coroutine utilizes the `SunCalculator` to dynamically query the exact local sunrise and sunset boundaries based on the user's defined latitude, longitude, and UTC offset. This prevents the simulation from wasting cycles tracing rays in the middle of the night. If the calculated sunrise and sunset times are identical (an edge case during polar nights at extreme latitudes), the simulation intelligently skips the entire day.

An aggressive computational culling strategy is further employed inside the hourly step loop. The engine skips any temporal step where both the incoming direct beam lux and the diffuse sky lux are calculated to be functionally zero (specifically < 0.1 Lux). By omitting these low-energy twilight steps where atmospheric scattering absorbs nearly all solar irradiance, *ChronoLux* saves millions of unnecessary Monte Carlo ray casts per simulated year, directly boosting overall performance.

During active simulation periods, the Coroutine explicitly uses the `yield return null;` statement. This instruction pauses the Coroutine's execution until the next engine frame, actively yielding the thread back to the Unity main loop. This guarantees that the UI remains fully responsive, the progress bar animates smoothly, and the TDR watchdog is appeased.

5.2.2. Mathematical Astrometry and Solar Ephemeris

Achieving metrological accuracy requires precise solar positioning at every hour of the year. The `SunCalculator.cs` class implements rigorous celestial mathematical models, determining not only the vector position of the solar disc but also evaluating atmospheric attenuation to yield physically accurate `BeamLux` and `DiffuseLux` values.

The first step converts the local simulation time into a Julian Day (JD) epoch, calculating Julian centuries T from J2000.0. The geometric mean longitude (L_0) and mean anomaly (M) of the sun are given by:

$$L_0 = (280.46646 + T(36000.76983 + T \times 0.0003032)) \pmod{360^\circ} \quad (5.1)$$

$$M = 357.52911 + T(35999.05029 - 0.0001537 \times T) \quad (5.2)$$

To account for Earth's orbital eccentricity, the Equation of Center (C) is derived using Fourier series expansion, which corrects the geometric mean longitude to the apparent longitude (λ_{app}):

$$C = \sin(M)(1.914602 - 0.004817T) + \sin(2M)(0.019993) + \sin(3M)(0.000289) \quad (5.3)$$

The Mean Obliquity of the Ecliptic (ϵ_0) is corrected for nutation to calculate the Solar Declination (δ), defining how far the sun resides above or below the celestial equator:

$$\delta = \arcsin(\sin(\epsilon) \sin(\lambda_{app})) \quad (5.4)$$

With δ established, the Equation of Time (EqT) maps solar time to clock time. Combining EqT with the terrestrial longitude yields the True Solar Time (TST) and the Hour Angle (H):

$$H = \frac{TST}{4.0} - 180^\circ \quad (5.5)$$

Finally, the Solar Zenith Angle (θ_z) and Azimuth (ϕ) are strictly defined relative to the user's latitude (Φ):

$$\cos(\theta_z) = \sin(\Phi) \sin(\delta) + \cos(\Phi) \cos(\delta) \cos(H) \quad (5.6)$$

With the astrometric vector position fixed, the clear-sky direct beam irradiance (E_{beam}) is computed using the rigorous Kasten-Young formula for optical air mass (AM). As the sun approaches the horizon, the thickness of the atmosphere the light must penetrate increases non-linearly:

$$AM = \frac{1}{\sin(\alpha) + 0.50572(\alpha + 6.07995)^{-1.6364}} \quad (5.7)$$

$$E_{beam} = 127500 \times 0.7^{AM^{0.678}} \quad [\text{Lux}] \quad (5.8)$$

Where α is the solar altitude angle ($90^\circ - \theta_z$), and 0.7 acts as the baseline atmospheric transmittance coefficient for clear-sky conditions. For ambient lighting, a simplified Haurwitz clear-sky diffuse model calculates horizontal diffuse illuminance ($E_{diffuse}$), estimating atmospheric scattering based on the total potential solar constant versus direct atmospheric penetration. These precise `BeamLux` and `DiffuseLux` values are injected directly into the HLSL compute kernel for every single time step.

5.3. Scene Management and Hardware Interfaces

5.3.1. Asynchronous Runtime Mesh Parsing

Because *ChronoLux* is distributed as a compiled desktop metrology application, it cannot rely on pre-compiled Unity Scene assets. It must dynamically load arbitrary, user-provided `.obj` files at runtime from the local filesystem. Heritage artifacts, such as laser-scanned church interiors or high-resolution photogrammetry models, can easily exceed several million vertices. If a string-based parser attempted to read these files synchronously on the main Unity thread, the application would permanently freeze, and the OS would terminate it with an "Application Not Responding" failure.

As demonstrated in Appendix 9.2 (Listing 9.4), the `RuntimeModelLoader.cs` resolves this memory bottleneck by implementing a heavily threaded chunk parser. The reading of the file and the string parsing of vertices, normals, and UV coordinates occurs entirely inside a `Task.Run()` background thread executing on separate logical CPU cores. While this parse task runs in the background, the main thread loop evaluates a `while (!parseTask.IsCompleted)` condition, repeatedly calling `yield return null;` and updating the `ProgressBar` UI element via `UpdateProgress()`. Only once the massive memory arrays are fully populated is execution yielded back to the Unity main thread for the final `Mesh.SetVertices()` upload to VRAM and the subsequent `GameObject` generation. This architecture keeps the UI fully responsive during the heaviest data loads.

5.3.2. Dynamic Material Serialization and Hierarchy Keying

When a heritage artifact is loaded via the runtime loader, the original materials are completely stripped. Instead, the simulation maps physically validated presets (e.g., Brick, Hardwood, Antique Glass) onto the individual sub-meshes. This mapping is securely persisted in the `ChronoProject` JSON file.

To re-apply these materials safely across different application sessions, the `RuntimeModelLoader` uses a unique string keying mechanism. Relying purely on a `GameObject`'s name is insufficient because `.obj` files often contain duplicate component names like `Wall` or `Window_01`. The `GetObjectKey` function constructs a distinct hierarchical identifier:

```

1 private string GetObjectKey(GameObject go) {
2     if (go.transform.parent != null && go.transform.parent != modelRoot
3     )
4         return $"{go.transform.parent.name}/{go.name}";
5     return go.name;
6 }
```

This `RootName/MeshName` composite key guarantees that when the `ApplySavedMaterial` logic iterates over the loaded mesh components, it securely matches the geometry against the serialized `project.objectNames` array and retrieves the exact GUID for the `project.materialPresetNames`, completely avoiding material crossover errors.

5.3.3. Spatial Normalization

Imported geometries exhibit extreme variations in scale; some architectural models are constructed in millimeters, others in meters, and some geographic surveys in kilome-

ters. To prevent catastrophic floating-point precision loss inside the DXR kernel—which occurs when rays must intersect extremely large coordinate spaces—*ChronoLux* applies a strict spatial normalization pipeline via the `CenterAndScaleModel` function.

The algorithm calculates the global bounding box encapsulating all sub-meshes attached to the artifact root. It determines the maximum dimensional span:

$$MaxDim = \max(Width, \max(Height, Depth)) \quad (5.9)$$

A uniform scaling factor is then calculated: $Factor = 2.0/MaxDim$ (assuming $MaxDim > 0$). Every child vertex transform is then translated so the object’s mathematical centroid rests exactly at local $(0,0,0)$, and scaled uniformly by the *Factor*. This mathematical normalization guarantees that the entire heritage site, regardless of its original CAD scale, strictly occupies a 2.0-meter bounding box in World Space. Because the DXR ray tracer relies on 32-bit floating point math, keeping intersection geometry strictly within the $[-1.0, 1.0]$ bounds near the origin maximizes the IEEE 754 mantissa precision, preventing “shadow acne” and geometric clipping. The user’s semantic transformations (position and scale offsets loaded from the `ChronoProject` JSON) are then applied exclusively to the normalized root object.

5.3.4. Deterministic BVH Building

As highlighted in theoretical radiometry literature, Monte Carlo integration is susceptible to floating-point associativity errors. However, an even greater threat in a hybrid CPU-GPU engine is structural non-determinism. Unity’s `FindObjectsByType<Renderer>()` API does not guarantee a strictly deterministic return order across different operating systems or CPU architectures. If the mapping between a ray’s geometric `InstanceID` and its underlying physical `SimulationMaterial` shifts between runs, the dosimetric simulation loses all scientific reproducibility.

To resolve this, the `IrradianceBaker.cs` initialization enforces strict determinism by collecting all active `Renderer` components and explicitly sorting them using a deeply granular $O(n \log n)$ string-based ordinal comparison. The unique `SortKey` for every sub-mesh is constructed as follows:

$$SortKey = ScenePath | TransformHierarchy | Type | MaterialSlot \quad (5.10)$$

The `TransformHierarchy` is built by recursively ascending the parent tree and appending the exact `GetSiblingIndex()` padded to 6 digits (e.g., `000002:Wall/000000:Window`). This incredibly stable ordinal sort guarantees that the structural `Index` and `Vertex` buffers uploaded to the GPU, alongside the `_SimulationMaterials StructuredBuffer`, are mathematically identical and perfectly aligned across entirely different hardware platforms, ensuring absolute reproducibility in lux-hour accumulation.

5.4. Hardware-Accelerated Path Tracing (HLSL / DXR)

The true computational bottleneck of *ChronoLux* lies within the ray tracing logic. For every pixel in the artifact’s UV map, stochastic rays must be cast to evaluate the rendering equation over thousands of simulated hours.

5.4.1. Memory Management and DXR Acceleration Structures

DirectX Raytracing relies on a hardware-accelerated two-tier hierarchy: the Bottom-Level Acceleration Structure (BLAS) storing raw triangle data, and the Top-Level Acceleration Structure (TLAS) defining spatial instances of the BLAS. In *ChronoLux*, the TLAS is built and managed via Unity's `RayTracingAccelerationStructure`.

Memory is strictly managed to align with GPU 16-byte boundaries. Data blocks transferred to the graphics card must obey HLSL packing rules to prevent memory corruption. For example, the `MeshMetadata` struct is explicitly padded inside C# to ensure an exact 16-byte footprint:

```
1 [StructLayout(LayoutKind.Sequential)]
2 private struct MeshMetadata {
3     public uint vertexOffset; // 4 bytes
4     public uint indexOffset;  // 4 bytes
5     public uint hasGeometry;  // 4 bytes
6     public uint padding;      // 4 bytes (Forcing 16-byte alignment)
7 }
```

Similarly, the GPU expects `float4` structures for optimal SIMD (Single Instruction, Multiple Data) processing lanes. As a result, the `_NormalMap` texture utilizes all four channels, storing the geometric (x, y, z) coordinates in the RGB channels, while reserving the alpha channel (w) as a strictly binary coverage flag indicating valid geometry versus empty padding space.

The `SimulationMaterial` variables (reflectance, transmittance) are packed into `Vector4` arrays and mapped into the `_SimulationMaterials` `ComputeBuffer`. When the `IrradianceBaker` builds the TLAS using `rtas.AddInstance()`, it applies specific flags. Sub-meshes are assigned `RayTracingSubMeshFlags.ClosestHitOnly`. This hardware optimization disables the invocation of "Any-Hit" shaders during traversal, significantly improving silicon performance since the simulation only requires the absolute nearest geometric intersection point.

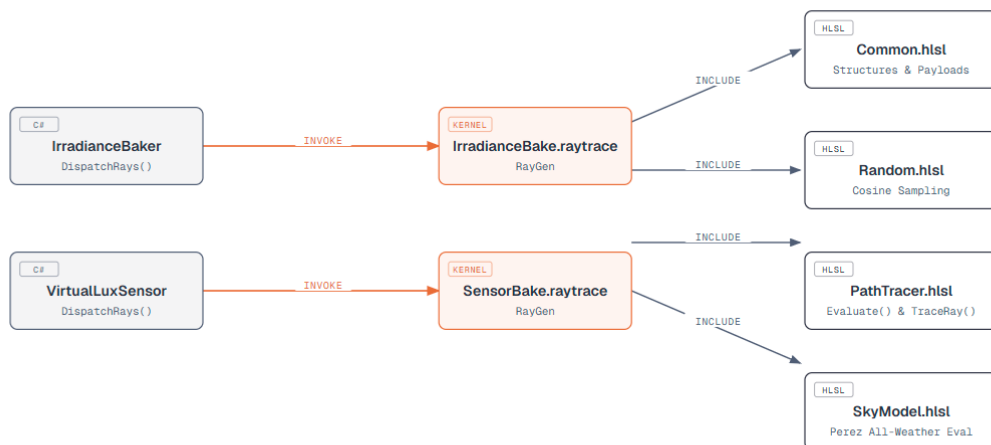


Figure 5.4: HLSL Compute Shader modular structure and inclusion tree.

5.4.2. Texture Space Dispatch Paradigm

ChronoLux drastically diverges from traditional screen-space video game graphics by utilizing Texture Space Ray Tracing. Instead of firing primary rays from the user’s virtual camera lens towards a 1080p screen grid (which would only accumulate dose for pixels currently visible to the user), the compute kernel maps threads directly to the two-dimensional (u, v) coordinates of the target mesh’s unrolled UV layout.

Require: $PosMap, NormMap, SunDir, N_{samples}, \Delta t$
for each thread $id \in DispatchThreadID$ **do**
 $\mathbf{p} \leftarrow PosMap[id].xyz$
 $\mathbf{n} \leftarrow NormMap[id].xyz$
 if $length(\mathbf{n}) < 0.1$ **then**
 return // Terminate thread (empty UV padding)
 end if
 $E_{dir} \leftarrow 0$
 $NdotL \leftarrow \mathbf{n} \cdot SunDir$
 if $NdotL > 0.0$ **then**
 $E_{dir} \leftarrow TRACEPATH(\mathbf{p}, SunDir, \dots, isDirectSun=true) \times NdotL$
 end if
 $E_{ind} \leftarrow 0$
 for $i = 1$ **to** $N_{samples}$ **do**
 $\omega_i \leftarrow GETCOSINEWEIGHTEDDIRECTION(\mathbf{n})$
 $E_{ind} \leftarrow E_{ind} + TRACEPATH(\mathbf{p}, \omega_i, \dots, isDirectSun=false)$
 end for
 $E_{total} \leftarrow E_{dir} + (E_{ind}/N_{samples}) \cdot \pi$
 $DoseMap[id] \leftarrow DoseMap[id] + E_{total} \cdot \Delta t$
 $DirectDoseMap[id] \leftarrow DirectDoseMap[id] + E_{dir} \cdot \Delta t$
 $IndirectDoseMap[id] \leftarrow IndirectDoseMap[id] + (E_{ind}/N_{samples}) \cdot \pi \cdot \Delta t$
end for

Figure 5.5: Pseudocode for Texture Space Ray Tracing dispatch separated into Direct and Stochastic Multi-Sampling paths.

A pre-pass baker uses the standard rasterization pipeline to write the world-space positions (P) and geometric normals (N) into `_PositionMap` and `_NormalMap` textures. As shown in Algorithm 5.5 (and detailed in Appendix 9.1), the ray generation kernel (`IrradianceBakeRayGen`) is dispatched across this 2D grid. If a pixel has a normal length near zero, it indicates empty void space in the UV padding, and the thread terminates instantly to save cycles. Otherwise, the kernel uses P and N as the origin and orientation for the evaluation of the rendering equation. This architecture makes light simulation completely detached from screen space, permanently baking energy data directly onto the architectural surface.

5.4.3. Pseudo-Random Sequence Generation and Hashing

Monte Carlo integration fundamentally relies on uniform random sampling across the target domain (the celestial hemisphere). If the random number generator exhibits clustering or spatial-temporal correlations, the rendering equation evaluation becomes

biased, manifesting as structural noise patterns rather than a uniform reduction in variance.

To resolve this inside the HLSL kernel without the overhead of tracking massive state arrays, *ChronoLux* implements a stateless hashing algorithm bound to the thread's pixel coordinate (x, y) and the active simulation time step (Δt) .

$$\text{baseSeed} = \text{Hash}(id.y \times \text{dims}.x + id.x + _FrameIndex) \quad (5.11)$$

By directly injecting `\_FrameIndex` (a sequentially increasing integer tracking the specific temporal step) into the core integer hash, the generated sequence of stochastic multi-samples shifts continuously throughout the year. As hundreds of time steps accumulate into the single `\_DoseMap` buffer, the high-frequency noise mathematically cancels itself out, allowing the Monte Carlo estimator to organically converge towards the true ground-truth irradiance integral.

5.4.4. Direct Sunlight and Stochastic Multi-Sampling Paths

To accurately evaluate the ambient light bouncing through the environment, *ChronoLux* distinctly splits the integration into two explicit paths inside the ray generation shader.

The first path handles **Direct Sun**. If the surface normal faces the solar vector (`nDotSun > 0.0`), a single specific ray is fired directly towards the analytical sun vector by calling `TracePath` with `isDirectSun = true`. Inside `PathTracer.hlsl`, direct rays are uniquely allowed to traverse through transparent geometry. If the direct ray hits a material, the hardware queries the `\_SimulationMaterials` buffer for `transmittance`. If transmittance is greater than zero (e.g., stained glass), the throughput multiplier is attenuated by the transmittance value, the ray origin is nudged explicitly forward, and the ray continues its traversal through the object towards the sun. If transmittance is zero, it acts as an opaque shadow caster, and the traversal immediately returns 0.0 lux.

The second path handles **Indirect Ambient Bouncing** via Stochastic Multi-Sampling. For every pixel, a `for`-loop generates N_{samples} pseudo-random vectors over the hemisphere utilizing Cosine-Weighted Importance Sampling (`GetCosineWeightedDirection`). These rays call `TracePath` with `isDirectSun = false`.

Unlike direct rays, indirect rays utilize the probabilistic **Russian Roulette** path termination technique. Russian Roulette provides an unbiased mathematical framework for terminating infinite recursive bounces without arbitrarily clipping light energy. At each bounce intersection, a random float $rv \in [0, 1)$ is generated from the hash sequence. Based on the material's scalar `reflectance` (R) and `transmittance` (T), the ray's fate is decided via cumulative probability thresholds:

1. **Reflection** ($rv < R$): The ray survives and generates a new cosine-weighted hemisphere vector.
2. **Transmission** ($rv < R + T$): The ray passes directly through the surface geometry without deviating its trajectory.
3. **Absorption** ($rv \geq R + T$): The ray is probabilistically absorbed by the material, and the path is immediately terminated.

Because the ray only continues its trajectory when it mathematically "survives" the material's properties, the throughput does not need to be manually divided by the reflection multiplier at every vertex—the probability of survival itself acts as the exact energy attenuation weighting factor. If an indirect ray escapes the bounding volume hierarchy (BVH) without being fully absorbed, it triggers the `EvaluateSky()` function,

tapping into the Perez Sky Model to fetch physically accurate atmospheric luminance. This deeply recursive behavior allows *ChronoLux* to accurately calculate highly occluded indirect lighting scenarios, such as light bouncing down a hallway or reflecting off a highly reflective marble floor onto a textile artifact.

5.5. Advanced Mathematical Models and Optimization

To ensure the scientific validity and computational feasibility of the simulation, several advanced mathematical paradigms and hardware optimizations are integrated into the *ChronoLux* engine.

5.5.1. Perez Sky Model Atmospheric Coefficients

While the direct solar disc dictates the intense beam irradiance, the vast majority of complex architectural ambient lighting is driven by diffuse sky scattering. *ChronoLux* evaluates this using the Perez All-Weather Sky Model (`SkyModel.hls1`), which models the highly anisotropic luminance of the celestial dome.

The engine implements the model for a clear sky with a Turbidity of $T = 2.0$, utilizing five empirically derived static coefficients:

- $a = -0.1058$: Dictates horizon brightening or darkening.
- $b = -0.0738$: Controls the luminance gradient near the horizon.
- $c = 1.8200$: Defines the relative intensity of the circumsolar corona.
- $d = -2.9744$: Controls the spatial width of the circumsolar region.
- $e = 0.1778$: Defines the backward scattered light opposing the sun.

Whenever an indirect stochastic ray misses all geometry, the zenith angle (θ) and the angular distance to the sun (γ) are calculated. The relative luminance is evaluated via the scattering indicatrix function:

$$f(\theta, \gamma) = \left[1 + a \exp\left(\frac{b}{\cos \theta}\right) \right] \times [1 + c \exp(d\gamma) + e \cos^2 \gamma] \quad (5.12)$$

The result is then normalized against the dynamically calculated `DiffuseLux` provided by the CPU astrometry system (via the normalization factor $E_{diffuse} \times 0.2$). This evaluates to a physically accurate spectral distribution that perfectly mimics real-world atmospheric scattering.

5.5.2. Hardware Traversal Mechanics (DXR and Face Culling)

The decision to build *ChronoLux* strictly upon DirectX 12 DXR fundamentally shifts the traversal bottleneck from software to raw silicon. Modern desktop GPUs feature dedicated Ray Tracing Cores that natively accelerate the traversal of the BVH tree and the complex Moeller-Trumbore ray-triangle intersection mathematical tests.

Furthermore, when the `TraceRay HLSL` intrinsic is called for direct sunlight shadows, a specific ray flag is injected into the payload structure: `RAY_FLAG_CULL_BACK_FACING_TRIANGLES`. Because direct shadow rays only care about binary occlusion (is the sun visible or blocked?), and since architectural CAD models are typically closed manifolds (e.g., a wall has two sides), this hardware flag immediately discards intersection tests against any triangle facing away from the ray origin. This halves the computational burden on the RT Cores for

shadow evaluation. Crucially, this optimization is explicitly disabled for indirect multi-bounce rays, as light must be able to reflect off the interior back-faces of transparent geometry like display cases.

5.6. Asynchronous Data Telemetry and UI Integration

5.6.1. Automated Sensor Placement

In physical metrology, scientists place physical lux meters around the laboratory. In the digital twin, *ChronoLux* mimics this behavior by deploying `VirtualLuxSensor` objects. To guarantee objective baseline measurements, the `RuntimeModelLoader` automates the placement of an outdoor reference sensor array. When an environment model is loaded, the engine invokes `CreateOutdoorReferenceSensors`. This function iterates across all renderers to calculate a global master bounding box enclosing the entire heritage site. It derives the highest spatial point ($Y_{max} + 2.0\text{m}$) and the total geometric radius. Five sensors are then spawned automatically—one directly in the center, and four arranged in a complete perimeter circle. This ensures that regardless of the complexity of the CAD artifact below, there will always be unobscured sensors tracking the unadulterated baseline sunlight potential for that specific latitude and day.

5.6.2. Metrological Analytics Pipeline

True scientific metrology requires raw, extractable numerics. The quantitative availability of precise light dose metrics is critical for subsequent statistical analysis in Excel or Python. The application relies on an asynchronous, headless telemetry extraction pipeline to move high-resolution floating-point data from VRAM back to the CPU without stuttering the 60 FPS render cycle.

```

Require: DownsampleRT, DirectRT, IndirectRT
completedRequests  $\leftarrow$  0
Trigger AsyncGPUReadback.Request(DownsampleRT)
Trigger AsyncGPUReadback.Request(DirectRT)
Trigger AsyncGPUReadback.Request(IndirectRT)
while completedRequests < 3 do
    Wait asynchronously (Non-blocking)
end while
data  $\leftarrow$  Extract float arrays
MaxDose  $\leftarrow$  0,  $\mu \leftarrow$  0, CoverageCount  $\leftarrow$  0,  $N_{active} \leftarrow$  0
for  $i = 0$  to 65536 do // Iterating 256x256 buffer
    val  $\leftarrow$  data[i]
    if val > 0.0 then
         $N_{active} \leftarrow N_{active} + 1$ 
         $\mu \leftarrow \mu + val$ 
        if val > MaxDose then MaxDose  $\leftarrow$  val
        end if
        if val > Threshold then CoverageCount  $\leftarrow$  CoverageCount + 1
        end if
    end if
end for
 $\mu \leftarrow \mu / N_{active}$ 
CoveragePercent  $\leftarrow$  (CoverageCount /  $N_{active}$ )  $\times$  100

```

Figure 5.6: CPU-Side Asynchronous Readback loop computing total scene variance and coverage metrics without blocking the primary rendering thread.

Iterating sequentially over a 1024×1024 float array every single simulated hour on the CPU would devastate performance. To resolve this, `LightDoseSimulator.cs` utilizes hardware `Graphics.Blit` downsampling. Three heavily downsampled 256×256 RenderTextures (`_downsampleRT`, `_directDownsampleRT`, `_indirectDownsampleRT`) are generated natively on the GPU.

Instead of blocking the thread, the C# system invokes Unity's `AsyncGPUReadback.Request`. As demonstrated in Algorithm 5.6, this schedules a direct memory access (DMA) transfer. A callback closure tracks a `completed` counter. Only when the counter reaches 3 (signifying all three buffers have successfully arrived in system RAM) does the CPU retrieve the underlying float arrays: `req.GetData<float>().ToArray()`.

Once the float arrays are safely on the CPU, a highly optimized memory loop iterates over the 65,536 downsampled pixels. It computes the absolute maximum dose exposure, counts the number of strictly active pixels to derive the arithmetic mean (μ), and tracks exactly how many pixels have crossed the user-defined `coverageThreshold` (e.g., 50,000 Lux-Hours for delicate watercolor paintings). By maintaining distinct arrays for Direct and Indirect accumulation, the telemetry framework allows conservators to isolate exact sources of UV/Visible energy damage.

5.6.3. User Interface Toolkit and JSON Serialization Constraints

The graphical interface of *ChronoLux* is built utilizing Unity's modern UI Toolkit (UXML/USS), replacing the legacy Canvas immediate-mode GUI with a strict Model-

View-Controller (MVC) DOM-based architecture. This responsive framework ensures the interface scales flawlessly across 4K laboratory monitors.

Scientific reproducibility dictates that experimental parameters must be precisely savable. The `ProjectManager.cs` system employs a custom JSON serialization engine via `JsonUtility`. However, architecting this serialization pipeline presented a significant software engineering challenge: Unity’s native `JsonUtility` explicitly does not support the serialization of standard C# `Dictionary<TKey, TValue>` collections.

To bypass this engine limitation without resorting to heavy third-party JSON libraries (which would bloat the executable), the `ChronoProject` architecture implements a structural data flattening technique. Instead of saving a dictionary mapping a sub-mesh’s string key directly to its material GUID, the data is manually split into two perfectly synchronized, parallel lists:

```

1 [Serializable]
2 public class ChronoProject {
3     public string projectName;
4     public string artifactPath;
5
6     // Parallel arrays resolving Dictionary serialization limits
7     public List<string> objectNames = new List<string>();
8     public List<string> materialPresetNames = new List<string>();
9 }

```

When an experiment is saved, the `ProjectManager` iterates over the active simulation hierarchy, extracting the `RootName/MeshName` keys and pushing them into the `objectNames` array, while simultaneously pushing the corresponding Material GUID into the `materialPresetNames` array at the exact same index. When the project is later loaded, a reconstruction loop dynamically rebuilds the C# dictionary in local memory before applying it to the DXR `IrradianceBaker`. Due to the complexity of the data—including nested string-based hierarchy paths mapped to specific Material GUIDs and spatial roots—the JSON format provides a human-readable experimental manifest.

When an experiment is launched, the data flows in a strictly unidirectional pattern: the `LightDoseSimulator` calculates astrometric variables, updates its internal C# `Model`, and triggers UI property-changed events. The UI Controller intercepts these updates to refresh the DOM view. This architectural separation ensures that the intensive background math calculations never directly mutate the UI layout, maintaining a strict Model-View boundary.

5.7. Deep Engine Optimizations and Memory Integrity

Scaling a dosimetric ray tracer to process millions of rays per frame demands extreme optimization. Several low-level engine mechanics must be carefully orchestrated to maintain fidelity and hardware stability over an 8,760-hour simulation.

5.7.1. Ray-Origin Epsilon Offsetting and Floating Point Limits

A fundamental mathematical vulnerability of ray tracing architectures involves IEEE 754 floating-point truncation. When an intersection is identified by the BVH, the new ray origin is placed precisely on the mathematical surface of the triangle. Due to limited 32-bit float precision, the calculated point might mathematically fall slightly *inside* the geometry. If the next bounce ray is cast from this point, it will immediately

intersect the triangle it originated from, causing massive "shadow acne" artifacts that destroy dosimetric validity.

To prevent this, *ChronoLux* implements a strict normal-based epsilon offset inside `PathTracer.hls1`.

$$\mathbf{p}_{offset} = \mathbf{p}_{hit} + \mathbf{n}_{hit} \times 0.001 \quad (5.13)$$

By offsetting the origin exactly 1 millimeter (0.001 units in the normalized 2.0-meter coordinate space) along the geometric surface normal, the ray is pushed safely outside the bounds before the subsequent BVH traversal begins. Furthermore, for direct rays passing *through* transparent materials, a magnified offset (`RAY_BIAS * 5.0`) is explicitly applied to guarantee the ray punches completely through thin window panes without getting trapped internally at extreme grazing angles.

5.7.2. Enforcement of Helmholtz Reciprocity and Energy Conservation

The accuracy of the Monte Carlo integration is entirely dependent on the physical validity of the materials in the scene. The Rendering Equation dictates that an opaque surface cannot reflect more energy than it receives. For translucent materials (e.g., stained glass, thin Japanese paper), the sum of Reflectance (R) and Transmittance (T) must never exceed 1.0.

If $R+T > 1.0$, the mathematical system literally creates energy from nothing. In a recursive path tracer, this triggers exponential positive feedback loops—causing materials to glow radioactively and completely ruining the lux-hour accumulation data.

ChronoLux enforces this physical law strictly at the C# binding layer. Inside `Ir-radianceBaker.cs`, the `ClampEnergy` function operates via mathematical normalization rather than aggressive hard-clamping:

```
1 private static void ClampEnergy(ref float r, ref float t) {
2     r = Mathf.Clamp01(r);
3     t = Mathf.Clamp01(t);
4     float s = r + t;
5     if (s > 1f) { r /= s; t /= s; }
6 }
```

Rather than aggressively clamping Transmittance to zero in a destructive manner, dividing both scalar components by their sum (s) elegantly scales them proportionally down to a mathematically safe maximum 1.0 boundary. These verified floating-point limits are written directly into the `_SimulationMaterials` buffer, guaranteeing the GPU's throughput multiplier strictly decays and obeys the universal law of conservation of energy.

5.7.3. Thread Synchronization and CPU-GPU Data Dependencies

The hybrid architecture of *ChronoLux* relies on strict synchronization barriers between the host CPU and the device GPU. Because the C# Coroutine loop executes asynchronously relative to the GPU's command queue, race conditions are a severe threat. If the CPU advances the clock Δt and uploads new `SunDirection` uniforms while the GPU is still executing the ray tracing kernel for the previous hour, the resulting light integration will be utterly corrupted.

To mitigate this, *ChronoLux* employs a rigorous pipeline leveraging Unity's `CommandBuffer` architecture. Rather than calling `ComputeShader.Dispatch()` directly—which

is a hazardous "fire-and-forget" command—the `IrradianceBaker` enqueues all `SetRayTracingVectorParam` assignments and `DispatchRays` commands into a designated `CommandBuffer`.

```

1 var cmd = new CommandBuffer { name = "ChronoLux Dispatch" };
2 cmd.SetRayTracingAccelerationStructure(rayTracingShader, _ID_SceneRTAS,
   _rtas);
3 cmd.SetRayTracingVectorParam(rayTracingShader, _ID_SunDirection,
   sunDirection);
4 cmd.DispatchRays(rayTracingShader, "IrradianceBakeRayGen", width,
   height, 1, cam);
5 Graphics.ExecuteCommandBuffer(cmd);
6 cmd.Release();

```

Invoking `Graphics.ExecuteCommandBuffer` synchronously submits the exact state of the variables into the rendering pipeline, enforcing absolute temporal order. This ensures that the DXR kernels evaluate exactly the correct sun position and solar intensity for that specific hour before the Coroutine is allowed to advance the master calendar loop.

5.7.4. VRAM Memory Integrity and Cleanup Protocols

When conservators frequently swap high-resolution 3D projects inside the *ChronoLux* application, Gigabytes of VRAM are allocated to hold the `_doseMap` buffers, the `_rtas` acceleration structures, and the massive `_vertexBuffer` arrays. If this native GPU memory is not manually freed, the application will trigger an out-of-memory crash.

To prevent leaks, a rigorous unmanaged `CleanUp()` protocol is enforced in the `IrradianceBaker`. Every time a new simulation is initialized, or when the `MonoBehaviour` is destroyed, explicit `.Release()` functions are invoked on every `ComputeBuffer` and `RenderTexture`. This guarantees stable and sustained laboratory usage without degrading the performance of the host operating system.

5.7.5. Data Export Payload Generation and Mathematical Variance

At the end of every simulated chronological day, the high dynamic range accumulation buffer must be flushed to the hard drive as an `.exr` file. This is handled by reading the pixels of the active dose map directly into a `Texture2D` buffer using `ReadPixels`, encoding it natively to the EXR format via `ImageConversion.EncodeToEXR(tex, Texture2D.EXRFlags.OutputAsFloat)`, and executing a synchronous `File.WriteAllBytes` to the local disk.

Simultaneously, the vast telemetry cache populated by the `TakeAsyncSnapshot` pipeline is flushed to a rigorous CSV file via the `ExportSimulationDataCSV` coroutine. The engine executes profound mathematical derivation for every single saved frame. To calculate the `hourlyVariance` (σ^2) across the entire surface of the artifact without risking arithmetic overflow over millions of floating-point sums, the engine computes:

$$\sigma^2 = \left(\frac{\sum (\Delta_{dose})^2}{Area} \right) - \mu^2 \quad (5.14)$$

Where the variance is derived algebraically from the sum of squares minus the square of the mean (μ). The resulting metrics, including the exact Delta Dose contributions (from isolated direct beam penetration vs ambient indirect bouncing), are passed into a

`StreamWriter` and appended sequentially to the `SimulationMetrics.csv`. This structured export allows external scientists to process the damage thresholds through arbitrary data-science frameworks outside of the Unity ecosystem, officially bridging the computational pipeline with actionable conservation science.

Chapter 6. Testing and Validation

Together with the previous two chapters should take about 70% of the paper.

Chapter 7. User's Manual

In the section describing the installation procedure you should detail the hardware and software resources needed for installing and running the application, and a step by step description of how your application can be deployed/installed. An administrator should be able to perform the installation/deployment based on your instructions.

In the section for the user you should describe how to use the application from the point of view of a user with no inside technical information; this should be adorned with screen shots and a stepwise explanation of the interaction. Based on user's manual, a person should be able to install and use your product.

Should take 1 to 5 pages.

Chapter 8. Conclusions

This chapter should take one or two pages.

In this chapter you should include::

- A summary of your contributions/achievements,
- A critical analysis of the results achieved,
- A description of the possibilities of improvements/further development.

Bibliography

- [1] S. Michalski, “Damage to museum objects by visible radiation (light) and ultraviolet radiation (uv),” *Conference on lighting in museums, galleries and historic houses*, pp. 3–16, 1987. [Online]. Available: <https://www.canada.ca/en/conservation-institute/services/agents-deterioration/light.html>
- [2] CIE, “Control of damage to museum objects by optical radiation,” International Commission on Illumination, Tech. Rep. 157:2004, 2004. [Online]. Available: <https://cie.co.at/publications/control-damage-museum-objects-optical-radiation>
- [3] H. Entradas Silva, G. Coelho, and F. Henriques, “Climate monitoring in world heritage list buildings with low-cost data loggers: The case of the jerónimos monastery in lisbon (portugal),” *Journal of Building Engineering*, p. 101029, 10 2019. [Online]. Available: <https://doi.org/10.1016/j.job.2019.101029>
- [4] A. Chianese, F. Piccialli, and J. E. Jung, “The internet of cultural things: Towards a smart cultural heritage,” in *2016 12th International Conference on Signal-Image Technology & Internet-Based Systems (SITIS)*, 2016, pp. 493–496. [Online]. Available: <https://doi.org/10.1109/SITIS.2016.83>
- [5] J. T. Kajiya, “The rendering equation,” *SIGGRAPH Comput. Graph.*, vol. 20, no. 4, p. 143–150, Aug. 1986. [Online]. Available: <https://doi.org/10.1145/15886.15902>
- [6] M. Pharr, W. Jakob, and G. Humphreys, *Physically Based Rendering: From Theory to Implementation*. MIT Press, 2023. [Online]. Available: <https://books.google.ro/books?id=IQpmzwEACAAJ>
- [7] G. Ward Larson and R. Shakespeare, *Rendering with Radiance: the art and science of lighting visualization*. Morgan Kaufmann, 1998. [Online]. Available: <https://www.radiance-online.org/learning/documentation/book>
- [8] P.-P. Sloan, J. Kautz, and J. Snyder, “Precomputed radiance transfer for real-time rendering in dynamic, low-frequency lighting environments,” *ACM Trans. Graph.*, vol. 21, no. 3, pp. 527–536, Jul. 2002. [Online]. Available: <https://doi.org/10.1145/566654.566612>
- [9] P. Jouan and P. Hallot, “Digital twin: Research framework to support preventive conservation policies,” *ISPRS International Journal of Geo-Information*, vol. 9, no. 4, 2020. [Online]. Available: <https://www.mdpi.com/2220-9964/9/4/228>
- [10] F. Niccolucci, A. Felicetti, and S. Hermon, “Digital twins for cultural heritage,” *Heritage*, vol. 5, no. 4, pp. 2397–2414, 2022. [Online]. Available: <http://dx.doi.org/10.2139/ssrn.4274457>

- [11] J. Burgess, “Rtx on - the nvidia turing gpu,” *IEEE Micro*, vol. PP, pp. 1–1, 02 2020. [Online]. Available: https://www.researchgate.net/publication/339036275-RTX_On_-_The_NVIDIA_Turing_GPU
- [12] R. Perez, R. Seals, and J. Michalsky, “All-weather model for sky luminance distribution—preliminary configuration and validation,” *Solar Energy*, vol. 50, no. 3, pp. 235–245, 1993. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0038092X9390017I>
- [13] E. Veach and L. J. Guibas, “Optimally combining sampling techniques for monte carlo rendering,” in *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH ’95. New York, NY, USA: Association for Computing Machinery, 1995, pp. 419–428. [Online]. Available: <https://doi.org/10.1145/218380.218498>
- [14] J. Munkberg, J. Hasselgren, P. Clarberg, M. Andersson, and T. Akenine-Möller, “Texture space caching and reconstruction for ray tracing,” *ACM Trans. Graph.*, vol. 35, no. 6, Dec. 2016. [Online]. Available: <https://doi.org/10.1145/2980179.2982407>
- [15] I. Reda and A. Andreas, “Solar position algorithm for solar radiation applications,” *Solar Energy*, vol. 76, no. 5, pp. 577–589, 2004. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0038092X0300450X>

Chapter 9. Appendix

9.1. Core HLSL Path Tracing Implementation

The following code details the exact DirectX Raytracing implementation used within *ChronoLux* to evaluate the dosimetric simulation. The integration is explicitly split into direct sun evaluation and a secondary stochastic multi-sampling loop for indirect ambient bouncing.

```
1 [shader("raygeneration")]
2 void IrradianceBakeRayGen()
3 {
4     uint2 id = DispatchRaysIndex().xy;
5     uint2 dims = DispatchRaysDimensions().xy;
6     float4 bakedPos = _PositionMap[id];
7     float4 bakedNormal = _NormalMap[id];
8
9     if (bakedPos.w < 0.01 || length(bakedNormal.xyz) < 0.1) return;
10
11     float3 origin = bakedPos.xyz;
12     float3 normal = normalize(bakedNormal.xyz);
13
14     uint baseSeed = Hash(id.y * dims.x + id.x + _FrameIndex);
15     float totalIrradiance = 0.0;
16     float nDotSun = dot(normal, _SunDirection);
17
18     // 1. Direct Sun
19     float directIrradiance = 0.0;
20     if (nDotSun > 0.0)
21     {
22         uint sunSeed = baseSeed;
23         directIrradiance = TracePath(origin, _SunDirection, normal,
24         sunSeed, true, _SceneRTAS, _SunDirection, _BeamLux, _DiffuseLux) *
25         nDotSun;
26         totalIrradiance += directIrradiance;
27     }
28
29     // 2. Stochastic Multi-Sampling
30     float indirectIrradiance = 0.0;
31     uint numSamples = max(1, _SamplesPerPixel);
32     for (uint i = 0; i < numSamples; i++)
33     {
34         uint sampleSeed = Hash(baseSeed + i);
35         float3 randomDir = GetCosineWeightedDirection(normal,
36         sampleSeed, PI);
37         indirectIrradiance += TracePath(origin, randomDir, normal,
38         sampleSeed, false, _SceneRTAS, _SunDirection, _BeamLux, _DiffuseLux)
39         ;
40     }
```

```

36
37     float avgIndirect = (indirectIrradiance / (float)numSamples) * PI;
38     totalIrradiance += avgIndirect;
39
40     _DoseMap[id] += (totalIrradiance * _DeltaHours);
41     _DirectDoseMap[id] += (directIrradiance * _DeltaHours);
42     _IndirectDoseMap[id] += (avgIndirect * _DeltaHours);
43 }

```

Listing 9.1: IrradianceBake.raytrace: The primary ray generation kernel mapping Target Mesh UVs to stochastic integration over the hemisphere.

```

1 float TracePath(float3 origin, float3 direction, float3 surfaceNormal,
  inout uint seed, bool isDirectSun, RaytracingAccelerationStructure
  rtas, float3 sunDir, float beamLux, float diffuseLux)
2 {
3     float throughput = 1.0;
4     float3 currentOrigin = origin + surfaceNormal * RAY_BIAS;
5     float3 currentDir = direction;
6
7     [loop]
8     for (uint bounce = 0; bounce < MAX_BOUNCES; bounce++)
9     {
10         RayDesc ray;
11         ray.Origin = currentOrigin;
12         ray.Direction = currentDir;
13         ray.TMin = 0.001;
14         ray.TMax = 10000.0;
15
16         Payload payload;
17         payload.isOccluded = 1;
18         payload.reflectance = 0.0;
19         payload.transmittance = 0.0;
20         payload.hitDistance = 0.0;
21         payload.worldNormal = float3(0, 1, 0);
22
23         uint rayFlags = RAY_FLAG_FORCE_OPAQUE;
24         if (isDirectSun) rayFlags |=
RAY_FLAG_CULL_BACK_FACING_TRIANGLES;
25
26         TraceRay(rtas, rayFlags, 0xFF, 0, 1, 0, ray, payload);
27
28         if (payload.isOccluded == 0)
29         {
30             return EvaluateSky(currentDir, sunDir, beamLux, diffuseLux,
isDirectSun) * throughput;
31         }
32
33         if (isDirectSun)
34         {
35             float trans = saturate(payload.transmittance);
36             if (trans <= 0.0) return 0.0; // Blocked by opaque geometry
37
38             throughput *= trans;
39             // Use a larger bias to guarantee escaping the pane
geometry, especially at grazing angles
40             currentOrigin += currentDir * (payload.hitDistance +
RAY_BIAS * 5.0);

```

```

41         continue;
42     }
43
44     // Probabilistic event selection (Russian Roulette)
45     float rv = Random(seed);
46     float refl = saturate(payload.reflectance);
47     float trans = saturate(payload.transmittance);
48
49     if (rv < refl)
50     {
51         float3 hitPoint = currentOrigin + currentDir * payload.
hitDistance;
52         currentDir = GetCosineWeightedDirection(payload.worldNormal
, seed, PI);
53         currentOrigin = hitPoint + payload.worldNormal * RAY_BIAS;
54     }
55     else if (rv < refl + trans)
56     {
57         currentOrigin += currentDir * (payload.hitDistance +
RAY_BIAS);
58     }
59     else
60     {
61         return 0.0;
62     }
63
64     if (throughput <= MIN_VISIBILITY) break;
65 }
66 return 0.0;
67 }

```

Listing 9.2: PathTracer.hlsl: The core recursive path tracing loop evaluating Russian Roulette absorption and transparent geometric traversal.

9.2. C# Orchestrator Implementations

The following code snippets detail the core C# CPU orchestration logic. Listing 9.3 demonstrates the dynamic chronological time-stepping Coroutine used to drive the simulation safely across the OS thread schedules.

```

1 private IEnumerator RunSimulationInternal() {
2     float deltaHours = stepSeconds / 3600f;
3     int totalDays = endDay - startDay + 1;
4     int currentDayIdx = 0;
5
6     string exportDir = Path.Combine(Application.dataPath, "../Exports",
DateTime.Now.ToString("yyyyMMdd_HH:mm:ss"));
7     Directory.CreateDirectory(exportDir);
8
9     for (int day = startDay; day <= endDay; day++) {
10         DateTime date = new DateTime(year, 1, 1).AddDays(day - 1);
11         currentDayIdx++;
12
13         DateTime sunrise = SunCalculator.FindSunrise(date, latitude,
longitude, utcOffset);
14         DateTime sunset = SunCalculator.FindSunset(date, latitude,
longitude, utcOffset);

```



```

15         if (sunrise == date && sunset == date) continue; // Polar night
16         fallback
17
18         for (DateTime localTime = sunrise; localTime < sunset;
19             localTime = localTime.AddSeconds(stepSeconds)) {
20             if (!_isSimulating) yield break;
21
22             ApplySunPosition(localTime);
23
24             // Optimization: Skip exact sunrise/sunset steps where
25             // there is functionally no light
26             if (CurrentBeamLux < 0.1f && CurrentDiffuseLux < 0.1f) {
27                 continue;
28             }
29
30             // Dispatch hardware raytracing for this specific hour
31             irradianceBaker.DispatchRays(CurrentSunDirection,
32             CurrentBeamLux, CurrentDiffuseLux, deltaHours, samplesPerPixel,
33             baker.PositionMap, baker.NormalMap);
34             completedSteps++;
35             currentProgress = ((float)currentDayIdx / totalDays) * 100f
36         ;
37
38         // Extract telemetry data for the CSV asynchronously
39         TakeAsyncSnapshot(day, localTime.ToString("yyyy-MM-dd HH:mm
40         "), currentAltitude, currentAzimuth, CurrentBeamLux,
41         CurrentDiffuseLux);
42
43         if (completedSteps % 10 == 0) {
44             ApplyDosePreview();
45         }
46         // Yield execution back to the OS and Unity Main Thread to
47         // maintain interactivity
48         yield return null;
49     }
50
51     // Flush the daily accumulation map to disk as a floating-point
52     EXR
53     ExportDailyMap(exportDir, day);
54 }

```

Listing 9.3: LightDoseSimulator.cs: The Coroutine driving dynamic time-stepping, asynchronous telemetry extraction, and synchronous EXR flushing.

Listing 9.4 showcases the background threaded parser utilized to load massively dense geometric heritage assets natively at runtime.

```

1 string path = Path.Combine(ProjectManager.ModelFolder, artifactFile);
2 UpdateProgress(0.1f, $"Loading artifact: {artifactFile}");
3
4 // 1. Offload heavy string parsing to a background CPU thread
5 var parseTask = Task.Run(() => {
6     if (!File.Exists(path)) return null;
7     return SimpleObjLoader.ParseObj(File.ReadAllText(path));
8 });
9
10 // 2. Yield the main thread to keep the "LOADING ASSETS" UI animation

```

```

smooth
11 while (!parseTask.IsCompleted) {
12     UpdateProgress(0.1f, "Parsing artifact geometry...");
13     yield return null;
14 }
15
16 if (parseTask.IsCompletedSuccessfully && parseTask.Result != null &&
    parseTask.Result.Count > 0) {
17     var dataList = parseTask.Result;
18     _artifactInstance = new GameObject(Path.GetFileNameWithoutExtension
    (artifactFile));
19
20     if (_artifactInstance != null) {
21         _artifactInstance.transform.SetParent(modelRoot);
22         _artifactInstance.transform.localPosition = Vector3.zero;
23         _artifactInstance.transform.localScale = Vector3.one;
24
25         // 3. Assemble the hierarchy on the main thread
26         foreach (var data in dataList) {
27             GameObject child = CreateGameObjectFromData(data,
    artifactMaterial);
28             if (child == null) continue;
29             child.transform.SetParent(_artifactInstance.transform,
    false);
30             child.transform.localPosition = Vector3.zero;
31             SetupArtifactPart(child, project);
32         }
33
34         SetupArtifactRoot(_artifactInstance);
35
36         // 4. Normalize the raw mesh geometry (Centering + Scaling
    children to a 2m box)
37         CenterAndScaleModel(_artifactInstance, 2.0f);
38
39         // 5. Apply user-defined transformations on the root
40         _artifactInstance.transform.localPosition = project.
    artifactPosition;
41         if (project.artifactScale != Vector3.zero)
42             _artifactInstance.transform.localScale = project.
    artifactScale;
43     }
44 }

```

Listing 9.4: RuntimeModelLoader.cs: Multi-threaded ‘.obj’ chunk parsing avoiding main-thread freezes, followed by strict spatial normalization constraints.